MySQL Connectivity

student_demo.py :-

student_demo.py

```
import mysql.connector
from mysql.connector import Error

DB_CONFIG = {
    "host": "localhost",
    "user": "swaraj",     # <-- change if different
    "password": "123",    # <-- change if different
    "database": "testdb"  # <-- change if different
}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)

def ensure_table_and_schema():
    """Make sure `student` table exists with the required columns.
       If it exists but lacks 'course' column, add it automatically.
    """
    con = None
    try:
        con = get_connection()
        cursor = con.cursor()
        # Which DB are we connected to?
        cursor.execute("SELECT DATABASE();")
```

```
    db_name = cursor.fetchone()[0]
    print(f"Connected to database: {db_name}")

    # Does table `student` exist?
    cursor.execute(
        "SELECT COUNT(*) FROM information_schema.tables WHERE table_schema=%s AND table_name=%s",
        (db_name, 'student')
    )
    exists = cursor.fetchone()[0] == 1

    if not exists:
        # Create table with correct schema
        cursor.execute("""
            CREATE TABLE student (
                id INT AUTO_INCREMENT PRIMARY KEY,
                name VARCHAR(50),
                age INT,
                course VARCHAR(50)
            )
        """)
        con.commit()
        print("Created table `student` with columns (id, name, age, course).")
        return

    # Table exists -> check columns
    cursor.execute(
        "SELECT column_name FROM information_schema.columns WHERE table_schema=%s AND table_name=%s",
        (db_name, 'student')
```

```
        )
        cols = [row[0].lower() for row in cursor.fetchall()]
        print("Existing columns in `student`:", cols)

        # If course column missing, add it
        if 'course' not in cols:
            cursor.execute("ALTER TABLE student ADD COLUMN course VARCHAR(50)")
            con.commit()
            print("Added missing column `course` to `student` table.")

    except Error as e:
        print("Error while ensuring table/schema:", e)
        raise
    finally:
        if con:
            con.close()

def insert_student(name, age, course):
    try:
        con = get_connection()
        cursor = con.cursor()
        cursor.execute(
            "INSERT INTO student (name, age, course) VALUES (%s, %s, %s)",
            (name, age, course)
        )
        con.commit()
        print(" Student added successfully!")
    except Error as e:
        print(" Error inserting student:", e)
```

```
    finally:
        if con:
            con.close()

def view_students():
    try:
        con = get_connection()
        cursor = con.cursor()
        cursor.execute("SELECT * FROM student ORDER BY id")
        rows = cursor.fetchall()
        if not rows:
            print(" No student records found.")
            return
        print("\n Student Records")
        print("-" * 50)
        for r in rows:
            print(f"ID: {r[0]}  Name: {r[1]}  Age: {r[2]}  Course: {r[3]}")
        print("-" * 50)
    except Error as e:
        print(" Error reading students:", e)
    finally:
        if con:
            con.close()

def search_student(student_id):
    try:
        con = get_connection()
        cursor = con.cursor()
        cursor.execute("SELECT * FROM student WHERE id=%s", (student_id,))
```

```
        row = cursor.fetchone()
        if row:
            print(f" Found: ID:{row[0]} Name:{row[1]} Age:{row[2]} Course:{row[3]}")
        else:
            print(" Student not found.")
    except Error as e:
        print(" Error searching student:", e)
    finally:
        if con:
            con.close()

def update_student(student_id, name, age, course):
    try:
        con = get_connection()
        cursor = con.cursor()
        cursor.execute(
            "UPDATE student SET name=%s, age=%s, course=%s WHERE id=%s",
            (name, age, course, student_id)
        )
        con.commit()
        if cursor.rowcount == 0:
            print(" No student updated (ID may not exist).")
        else:
            print(" Student updated successfully!")
    except Error as e:
        print(" Error updating student:", e)
    finally:
        if con:
            con.close()
```

```
def delete_student(student_id):
    try:
        con = get_connection()
        cursor = con.cursor()
        cursor.execute("DELETE FROM student WHERE id=%s", (student_id,))
        con.commit()
        if cursor.rowcount == 0:
            print(" No student deleted (ID may not exist).")
        else:
            print(" Student deleted successfully!")
    except Error as e:
        print(" Error deleting student:", e)
    finally:
        if con:
            con.close()

def menu():
    ensure_table_and_schema()  # critical: ensure schema before anything else
    while True:
        print("""
===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
""")
```

```
choice = input("Enter your choice: ").strip()
if choice == "1":
    name = input("Enter name: ").strip()
    age = input("Enter age: ").strip()
    course = input("Enter course: ").strip()
    if not name or not age.isdigit():
        print(" Invalid input. Name required and age must be a number.")
        continue
    insert_student(name, int(age), course)
elif choice == "2":
    view_students()
elif choice == "3":
    sid = input("Enter student ID to search: ").strip()
    if sid.isdigit():
        search_student(int(sid))
    else:
        print(" Invalid ID.")
elif choice == "4":
    sid = input("Enter student ID to update: ").strip()
    if not sid.isdigit():
        print(" Invalid ID.")
        continue
    name = input("Enter new name: ").strip()
    age = input("Enter new age: ").strip()
    course = input("Enter new course: ").strip()
    if not age.isdigit():
        print(" Age must be a number.")
        continue
    update_student(int(sid), name, int(age), course)
```

```
        elif choice == "5":
            sid = input("Enter student ID to delete: ").strip()
            if sid.isdigit():
                delete_student(int(sid))
            else:
                print(" Invalid ID.")
        elif choice == "6":
            print(" Exiting... Goodbye Swaraj!")
            break
        else:
            print(" Invalid choice. Try again.")

if __name__ == "__main__":
    menu()
```

```
swaraj@swaraj-VirtualBox:~/DBMSL$ python3 student_demo.py
Connected to database: testdb
Existing columns in `student`: ['id', 'name', 'age']
Added missing column `course` to `student` table.

===== Student Management System =====
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
```

Enter your choice: 1

Enter name: Swaraj Gondchawar

Enter age: 22

Enter course: BE

Student added successfully!

===== Student Management System =====

1. Add Student

2. View Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 1

Enter name: Mayur Shinde

Enter age: 23

Enter course: BE

Student added successfully!

===== Student Management System =====

1. Add Student

2. View Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 2

Student Records

ID: 1 Name: Swaraj Age: 21 Course: None

ID: 2 Name: Amit Age: 22 Course: None

ID: 3 Name: Priya Age: 20 Course: None

ID: 4 Name: Swaraj Gondchawar Age: 22 Course: BE

ID: 5 Name: Mayur Shinde Age: 23 Course: BE

===== Student Management System =====

1. Add Student

2. View Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 3

Enter student ID to search: 4

Found: ID:4 Name:Swaraj Gondchawar Age:22 Course:BE

===== Student Management System =====

1. Add Student

2. View Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 4

Enter student ID to update: 1

Enter new name: Mahesh Rathod

Enter new age: 22

Enter new course: BE

Student updated successfully!

===== Student Management System =====

1. Add Student

2. View Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 5

Enter student ID to delete: 2

Student deleted successfully!

===== Student Management System =====

1. Add Student

2. View Students

3. Search Student

4. Update Student

5. Delete Student

6. Exit

Enter your choice: 6

Exiting... Goodbye Swaraj!